Run Builds, Not Risks: Isolation in CI/CD Workflows

Sofia Vaz*, Vitor A. Cunha† and Rui L. Aguiar‡
Instituto de Telecomunicações, Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática, Universidade de Aveiro
Email: *sofiateixeiravaz@ua.pt, †vitorcunha@ua.pt, ‡ruilaa@ua.pt

***Abstract*—CI/CD pipelines have become a critical component of modern software development, however, they also execute untrusted code, include uncontrolled third-party dependencies, and build at high frequency. Despite their central role, many CI/CD systems still rely on weak or ad-hoc isolation mechanisms, meaning that their execution environments, credentials and infrastructure are especially vulnerable to malicious activity. Moreover, since CI/CD security mostly rests on configuration, if misconfigured, these systems can be even more fragile. To address these challenges, we propose an approach that forces execution isolation. We implement a prototype CI/CD execution environment, leveraging Jenkins, that utilizes snapshot-based execution using lightweight virtualization. By reusing snapshots across pipeline runs, snapshot creation costs are minimized, allowing isolation to be enforced with negligible runtime overhead in steady-state operation. We evaluate our approach using representative CI/CD workloads and show that snapshot-based isolation significantly improves isolation guarantees while maintaining performance comparable to existing CI/CD execution models. Our results demonstrate that snapshot-based execution is a practical and effective foundation for securing modern CI/CD pipelines without sacrificing efficiency.**

***Index Terms*—CI/CD, Isolation, Security, Snapshots, Pipelines**

I. Introduction

Continuous Integration and Continuous Deployment (CI/CD) pipelines are a fundamental component of modern software development [1]. By automating build, test, and deployment processes, CI/CD enables rapid iteration, improves developer productivity, and has become a critical part of today's software supply chain [2]. In many organizations, these pipelines act as the primary integration point for source code, dependencies, build tools, and deployment targets, concentrating a substantial portion of the overall development workflow. As a consequence, the correctness, reliability, and security of CI/CD systems have a direct impact on the integrity of released software.

However, existing CI/CD systems often suffer from systemic security and reliability shortcomings [3]. Pipeline execution environments are commonly over-privileged and weakly isolated, while their behavior is largely driven by complex and error-prone configurations [4]. These characteristics make CI/CD pipelines particularly vulnerable to misconfigurations, privilege escalation, and supply-chain attacks [5].

In many organizations, isolation settings, runner topologies, and credential scoping evolve organically over time, shaped by convenience and legacy constraints rather than by an explicit security model. As a result, even when individual controls are in place, it is difficult to reason about the effective trust boundaries of a CI/CD setup or to predict how a compromised workflow might interact with shared infrastructure, cached state, or other concurrent jobs.

Rather than continuously patching individual vulnerabilities or adding ad-hoc security mechanisms, this paper argues for a different approach: enforcing isolation as a design principle in CI/CD workflows. By strictly isolating pipeline execution, it is possible to reduce the attack surface, limit the impact of compromised components, and provide stronger guarantees about security. Additionally, the fact that this is forced isolation, instead of configured isolation, means that there is less room for human error, which is a common source of vulnerabilities in CI/CD systems.

This paper explores enforced isolation as a foundational design principle for CI/CD pipelines, proposes a snapshot-based execution model that treats workflow environments as explicit, reusable artifacts, and demonstrates a Jenkins [1]-based prototype that integrates this model into a realistic pipeline setup. It further evaluates the approach across three execution modes — non-isolated, fully isolated, and snapshot-reusing "semi-isolated" workflows — showing that enforced isolation can significantly constrain the impact of malicious or misconfigured jobs while keeping steady-state runtime overhead close to that of conventional, non-isolated executions.

After this section, section II explores the current state of the art in CI/CD security and isolation techniques. Then, section III presents the proposed architecture for isolated CI/CD workflows. Afterwards, section IV details the implementation of a prototype system, and section V evaluates the effectiveness and performance impact of the proposed approach. The results are then discussed in section VI, and section VII concludes the paper and outlines directions for future work.

This study was funded by the European Union's Horizon Europe research and innovation program through the project EXIGENCE under grant agreement No. 101139120. Co-Funded from the European Union's Erasmus+ Capacity Building in the field of Higher Education Programme under Grant Agreement No 101179818. Neither the European Union nor the granting authorities can be held responsible for the views and opinions expressed in the present document.

[1] https://www.jenkins.io/

II. State of the Art

Research and industry practice have produced a wide range of mechanisms to improve the security of CI/CD pipelines. Most contemporary platforms—such as GitHub Actions [2], GitLab CI [3], Jenkins, and cloud-native CI services—provide isolation primarily through virtual machines or containers. While these mechanisms offer some degree of separation between jobs, in practice they are often configured with broad privileges, shared caches, persistent credentials, and reusable runners, which weakens their security guarantees [6], [7], [8]. Additionally, they may even not be configured at all, as bare-metal execution remains a common option. Studies of real-world CI/CD deployments have repeatedly demonstrated that misconfigurations, excessive permissions, and implicit trust relationships are widespread and exploitable [9].

A substantial body of work focuses on detecting vulnerabilities in pipeline configurations. Static analysis tools and linters have been proposed to identify insecure practices in pipeline definitions, such as over-scoped tokens [10], unsafe triggers [11], or the use of untrusted and/or vulnerable third-party tools [12]. Other approaches aim to harden pipelines through policy enforcement, for example by applying organizational security policies via centralized governance frameworks [13]. While these techniques can reduce accidental misconfigurations, they fundamentally rely on the correctness of complex policies and do not eliminate the underlying structural weaknesses of the execution model.

Another line of research addresses supply-chain security within CI/CD. Efforts such as provenance tracking [14] and frameworks like Supply-chain Levels for Software Artifacts (SLSA) [15] aim to improve traceability and trust in build outputs. These mechanisms provide valuable guarantees about the integrity of produced artifacts, but they typically assume the correctness and trustworthiness of the pipeline execution environment itself. If the execution environment is compromised or overly permissive, such guarantees can be undermined.

Sandboxing techniques and stronger isolation primitives have also been explored. Some systems leverage microVMs [16], or sandboxing [4] to confine untrusted build steps. While promising, these approaches are often treated as optional hardening features rather than as foundational design principles. As a result, they are rarely integrated end-to-end across the entire pipeline and its associated tooling.

Overall, the state of the art remains characterized by incremental defenses layered atop fundamentally permissive and complex pipeline architectures. Existing solutions tend to focus on detecting or mitigating specific classes of vulnerabilities rather than rethinking the execution model itself. This gap motivates a shift toward isolation-first CI/CD design, where strong separation between components is not an optional feature but a core architectural property.

[2]https://github.com/features/actions

[3]https://docs.gitlab.com/ci/

[4]https://plugins.jenkins.io/cloudshell-sandbox/

III. Concept

Building on the objective of making isolation the default rather than an optional configuration, this section presents the architecture of the proposed CI/CD system. The architecture is grounded on the principle that each workflow execution should occur inside an environment that closely replicates the machine on which the workflow would otherwise run, while remaining fully isolated from the host system and other workflows. Rather than relying on standardized runner images, the system captures and reproduces the concrete execution environment itself.

A. Architecture Overview

The architecture leverages containerization mechanisms to clone the execution machine and run the workflow within this cloned environment. Each workflow execution is sandboxed inside a self-contained instance that mirrors the original system in terms of operating system, installed software, and configuration. This approach preserves environmental fidelity and provides strong isolation guarantees, as the cloned environment is fully separated from the host and from other executions.

A central orchestration component coordinates the lifecycle of workflow executions. When a workflow is triggered, the orchestrator provisions an isolated environment by cloning the machine, transfers the workflow inputs into that environment, and initiates execution. Upon completion, artifacts and logs are collected, and the isolated environment is destroyed, ensuring that any state created during execution does not persist beyond the lifetime of the instance. As a result, executions are structurally prevented from interfering with one another.

Crucially, the architecture treats execution environments as explicit, identifiable artifacts. Each cloned environment can be tagged, versioned, and referenced by workflows. This enables users to intentionally reuse specific environments across executions when desired. For example, a workflow may declare that it should run against a particular environment image, ensuring that multiple runs share the same underlying system state. This capability is valuable not only for performance reasons, but also for reproducibility, debugging, and auditability: builds can be tied to concrete environment versions, failures can be reproduced under identical conditions, and changes in behavior can be attributed to explicit changes in the environment.

Importantly, even when an environment image is reused, each execution still takes place inside an ephemeral instance derived from that image and is destroyed after completion. Environment reuse therefore does not imply shared runtime state between executions, but rather controlled reuse of a well-defined environment snapshot. This preserves the isolation guarantees of the system while providing users with a practical and expressive way to manage execution environments.

This design directly addresses the limitations of existing systems where isolation is treated as an optional hardening feature and where environment management is often implicit and opaque. For instance, in many CI/CD pipelines, a build may inadvertently reuse dependencies from a previous job on

the same runner, leading to inconsistent or unreproducible results. In conventional CI/CD systems, users frequently rely on mutable runners or ad-hoc configuration to approximate environment stability, which makes behavior difficult to understand and easy to misconfigure. In contrast, the proposed architecture exposes environments as explicit objects within the system, making both isolation and reproducibility structural properties rather than emergent side effects.

By construction, this architecture limits the impact of compromised workflows, prevents persistence across executions, and reduces reliance on fragile configuration-based security. At the same time, it provides users with practical mechanisms to control and reuse execution environments in a principled manner.

B. Threat Model and Assumptions

The proposed architecture assumes a CI/CD setting in which workflow definitions and build scripts may contain untrusted code, either due to contributions from external collaborators, compromised dependencies, or malicious changes in version-controlled repositories. The attacker is therefore modeled as having the ability to execute arbitrary commands within the context of a pipeline step, but not to compromise the underlying operating system or container runtime before the workflow is launched.

The workflow orchestrator, the worker host operating system, and the snapshot creation mechanism are considered trusted components that enforce the intended isolation properties. In particular, we assume that the process that captures the filesystem snapshot and turns it into an image is not under the control of the attacker, and that image tags cannot be rewritten or forged without detection. Network infrastructure and storage backing the CI/CD platform are also assumed to follow the configured isolation and access-control policies.

Within this model, the adversary's primary goals include exfiltrating secrets, tampering with build artifacts, modifying system configuration to persist across runs, and interfering with other workflows executing on the same infrastructure. Our design focuses on limiting these goals by ensuring that each workflow executes in an ephemeral environment whose state is discarded at the end of the run, thereby preventing persistent modifications to both system and workflow files. We also want to ensure that the execution of workflows does not interfere with one another, even when they are running concurrently on the same machine, by enforcing strong isolation between them.

IV. Implementation

This section describes the proof-of-concept implementation developed to instantiate the proposed architecture and to enable experimental comparison between different degrees of isolation. The implementation is intentionally lightweight and focuses on architectural behavior rather than performance optimization. It is realized as an extension of an existing Jenkins-based CI/CD setup, using Podman as the container runtime and filesystem-level cloning to emulate machine capture.

The choice of Jenkins is motivated by its widespread adoption and its representativeness of conventional CI/CD systems that rely on direct script execution and weakly isolated agents. Rather than replacing Jenkins, the prototype is built around it in order to demonstrate how different execution models can be realized within the same orchestration framework. This allows isolation to be evaluated as an architectural property rather than as a platform-specific feature.

A. Execution Modes

The prototype supports three execution modes that correspond to different points in the isolation spectrum: a baseline mode without isolation, a fully isolated mode in which the execution environment is generated as part of the workflow, and a reuse mode in which a pre-existing environment image is reused.

In the baseline mode, Jenkins executes the workflow script directly on the host machine, using the standard execution model provided by the platform. This corresponds to the common configuration found in many real-world deployments, where jobs run on bare-metal agents or long-lived virtual machines. This mode serves as a reference point and represents the absence of architectural isolation. It is vital to note, however, that in all of these modes Jenkins' "sandbox" mode was enabled. Thus, this baseline should include some level of protection, and should approximate a typical real-world setup more closely.

In the fully isolated mode, the workflow itself includes the creation of an execution environment. Concretely, the system captures the current state of the machine, which is then used as the basis for constructing a container image. Podman is used to build and run a container from this image, and the workflow script is executed inside the resulting container. In this configuration, the execution environment is generated dynamically and is tightly coupled to the machine state at the time of execution. Each run produces a fresh image and executes in a container derived from that image, closely reflecting the architectural model in which each workflow executes inside a cloned environment.

In the reuse mode, the system reuses a previously generated image rather than creating a new one for each run. The workflow specifies which image should be used, and Jenkins launches a Podman container from that image to execute the script. This mode reflects the architectural feature described earlier in which execution environments are treated as explicit artifacts that can be referenced, versioned, and reused. Although the image is reused, each execution still takes place inside a fresh container instance that is destroyed after completion, preserving runtime isolation while avoiding the overhead of regenerating the image.

Across all three modes, Jenkins remains responsible for orchestrating execution, triggering runs, and collecting outputs. The primary difference lies in where and how the workflow script is executed: directly on the host, inside a container built from a newly generated image, or inside a container derived from an existing image.

B. Environment Capture and Container Execution

Environment cloning in the prototype is implemented using a simple mechanism: the filesystem of the execution machine is archived into a tarball, which is then used to construct a container image. While this approach does not capture every aspect of a running system, it is sufficient to replicate the user environment, including installed tools, libraries, and configuration files. This choice reflects the goal of demonstrating architectural feasibility rather than achieving perfect system-level fidelity.

Podman is used as the container runtime for both building images and executing containers. From the perspective of the Jenkins pipeline, container execution is treated as an implementation detail, as the workflow script is simply executed inside the container rather than on the host. In theory, other container runtimes or virtualization technologies could be used to achieve similar isolation properties.

C. Workload Design

The workload executed within the pipeline is intentionally minimal. The test script used consists solely of a fixed-duration sleep of ten seconds. This design choice is deliberate. The purpose of the prototype is not to evaluate the performance of build workloads, but to isolate and observe the overhead and behavioral differences introduced by the execution model itself. By keeping the workload computationally trivial and constant across all runs, any differences in execution time or behavior can be attributed primarily to the isolation mechanism rather than to variability in the workload.

D. Summary

The resulting implementation concretely instantiates the proposed architecture in a familiar CI/CD environment, with three Jenkins-based execution modes that enable direct comparison while showing that isolation-first execution can be achieved using existing tools and explicit, user-controlled environment snapshots. An overview of the implementation is present in Figure 1.

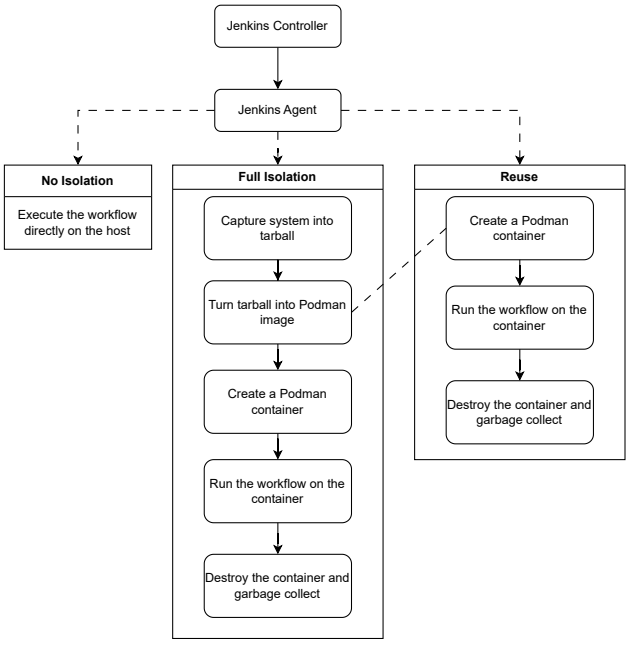

Fig. 1. Implementation overview

V. Results

In this section we present the evaluation results of our proposed snapshot-based isolation mechanism for CI/CD workflows.

A. Experimental Setup

To test the performance impact and isolation guarantees of our approach, we set up a series of experiments using two different machines. These will be henceforth be refferred to as the *Jenkins Machine* and the *Worker Machine*. The Jenkins Machine is responsible for hosting the Jenkins server, which orchestrates the execution of the CI/CD workflows. The Worker Machine is responsible for executing the workflows, utilizing our snapshot-based isolation mechanism. For all intents and purposes, the worker was, theoretically, not necessary, as Jenkins can run the workflows on the same machine it is hosted on. However, in order to better simulate a real-world scenario, and to avoid interference between the Jenkins server and the workflow executions, we opted to use a separate machine for running the workflows.

Both machines have the same specifications. They both run Ubuntu 24.04 as the operating system, and have 4 vCPUs. They also have 8 GB of RAM and 64 GB of storage. Both of these machines are virtual, and are hosted on an OpenStack instance. That can lead to some variability in the results, as the underlying physical hardware is shared between multiple virtual machines, which can lead to resource contention.

B. Experiments

As was mentioned previously, there are three different isolation levels that we are testing: No Isolation, Semi-Isolation, and Full Isolation. For each of these isolation levels, we designed a series of experiments to evaluate both the isolation guarantees and performance impact provided by our snapshot-based approach.

1) Isolation: To test the isolation guarantees provided by our snapshot-based approach, we designed a series of experiments aimed at attempting to breach the isolation boundaries set by our system.

We created a set of malicious workflows designed to perform unauthorized actions, such as reading and modifying files from other workflows and overall system files. In order to not interfere with the host system, the modified “system file” was created, being a simple text file with a location and permissions similar to system files. The path of the file is previously known to the malicious workflow, however, this is moreso to verify if an attacker is able to access the location, rather than to

simulate a real-world attack. If the attacker is even able to access the file, it is expected to assume that they will be able to access and change other system files as well.

Regarding Workflow Files, exploitability means being able to read and/or write in the directory `/var/jenkins/workspace/test`, which means being able to modify the files relating to the execution of another workflow. Regarding System Files, it means being able to read `/root/.ssh/authorized_keys`, which simulates the ability to read system files. In regard to the modified file, it is located at `/var/log`, a vital system directory, and writing into which would simulate the ability to delete logged malicious activity. Modification, in this case, means that it is possible to create and modify a file in the respective directory.

After testing, it was found that it is always possible to read system and workflow files, regardless of the isolation level. This is expected, as the snapshot-based isolation mechanism does not prevent read access to files. However, this is manageable, as during the snapshot creation process, it is possible to ignore certain directories. The fact that reading is always possible is not ideal, as it may lead to information leakage. Thus, steps should be taken to mitigate this risk, such as ignoring sensitive directories during snapshot creation. In regard to modification, modification is only persistent when there is no isolation. This is a significant result, as it demonstrates that our snapshot-based isolation mechanism is effective at preventing unauthorized modifications to both system and workflow files. Further exploration of these results and their implications is present in section VI.

A critical consequence of this architecture and these results is that execution environments are ephemeral: modifications within a workflow execution cannot persist into subsequent runs. This directly addresses the dependency contamination limitation identified in section III, where conventional systems risk state carryover between jobs.

2) Performance: Performance in this case is measured in terms of the time it takes to execute a given workflow. As mentioned previously, there is a dummy workflow that is used for testing purposes. This workflow will always run for 10 seconds, regardless of the isolation level. However, the question here is: how much overhead does our snapshot-based isolation mechanism introduce?

We executed the dummy workflow several times on each isolation level, measuring the time it took to complete each execution. Because the underlying shared infrastructure introduces some variability unrelated to the typical performance of the solution we have removed these outliers, but clearly state how many values were removed out of how many valid results. Our outlier definition follows that of a typical box plot, that is, the values not within the range of $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$, where Q_1 is the first quartile, Q_3 is the third quartile, and $\mathrm{IQR} = Q_3 - Q_1$ is the interquartile range.

Regarding the non-isolated environment, it took on average 12.216 seconds to execute the workflow, with a standard deviation of 0.081 seconds. There were a total of 643 valid results, with 1 outlier removed. Regarding the reuse environment, the setup time took on average 390.638 seconds, with a standard deviation of 23.387 seconds. There were a total of 442 valid results, with 5 outliers removed. The runtime of the reuse environment took on average 13.377 seconds to execute the workflow, with a standard deviation of 0.036 seconds. There were a total of 597 valid results, with 29 outliers removed. Finally, regarding the fully isolated environment, it took on average 582.543 seconds to execute the workflow, with a standard deviation of 16.486 seconds. There were a total of 242 valid results, with 13 outliers removed.

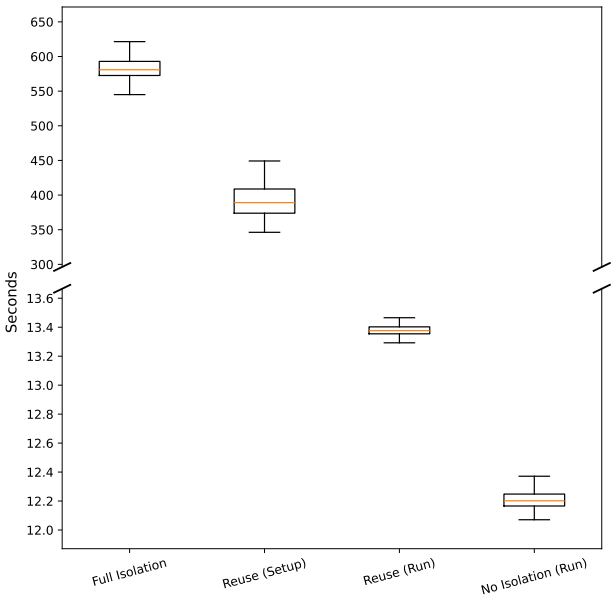

Fig. 2. Isolation Level Timing Results Comparison

Moreover, it was necessary to measure an additional time here – that of the setup of the reuse environment. The goal of the reuse environment is to use pre-existing snapshots of the system, however, it is vital to also measure how long the setup takes. These results are shown in Figure 2, which compares both the runtime of the no isolation and partial isolation methods, as well as the setup time of the reuse environment with the execution time of the fully isolated environment.

As is visible, the time it takes to execute the workflow is similar when there is no isolation and when there is partial isolation – the difference is 1 second. Additionally, the fully isolated environment takes significantly longer to execute the workflow, as expected. This is due to the fact that a fully isolated environment requires the creation of a new snapshot for each workflow execution, which is a time-consuming process. The same applies to the setup time of the reuse environment. However, it is vital to note that the setup is much quicker, taking a full three minutes less than the full isolation execution time.

A further discussion of these results, and what they imply, is present in section VI.

Additionally, RAM usage was measured, to verify if there were any significant differences between the different isolation

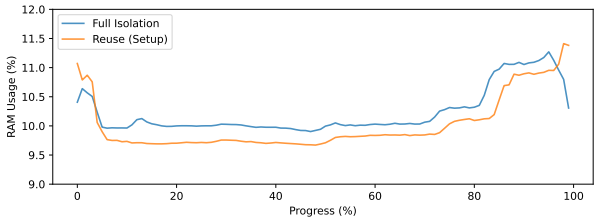

Fig. 3. RAM Usage Comparison: Full Isolation vs Reuse (Setup)

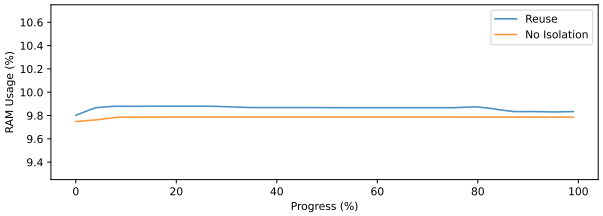

Fig. 4. RAM Usage Comparison: Reuse vs No Isolation

levels. RAM was chosen as the resource to be measured, as it is the resource that can most directly affect scaling, which, if the isolation methods increase its usage, could be a concern for performance. This was done by running the workflow (or setup) at least 150 times, and measuring the RAM usage, independently, every second. The workflow orchestrator would give the order for the workflow to run, but, at the worker level, there would be a separate process responsible for measuring the RAM usage every second, and sending the results to the orchestrator. RAM usage was measured using `psutil`, a Python library that provides an interface for retrieving information on system utilization (CPU, memory, disks, network, sensors) and running processes. In our case, our focus was gathering the percentage of RAM in use at the moment, which was measured using `psutil.virtual_memory().percent`.

To account for differences in execution duration, RAM utilization time series were temporally normalized by mapping each trace to a standardized time axis (0-1) and resampled via linear interpolation to 100 samples. The mean RAM usage profile was then computed by averaging across runs at each normalized time point. These results are shown in Figure 3 and Figure 4. There is some expected variability in the results, as the underlying shared infrastructure can lead to resource contention, which can affect RAM usage. However, we believe that the general trends shown in the figures are still valid, as they are based on a large number of runs, which should help to mitigate the effects of variability.

As is visible in Figure 3, the RAM usage of the fully isolated environment is slightly higher, at times, than the RAM usage of the reuse environment during setup. Additionally, in Figure 4, the RAM usage of the reuse environment during runtime is slightly higher than the RAM usage of the no isolation environment, but negligibly so.

A further discussion of these results, and what they imply, is present in section VI.

VI. Discussion and Limitations

As was visible in section V, the performance results indicate that the slowest system is that which includes the simultaneous creation of the image and its immediate usage. This is expected, as creating a snapshot is a time-consuming process. However, it is vital to note that the reuse environment, when pre-setup, has a performance very close to that of the no isolation environment. This indicates that, with proper management of snapshots, it is possible to achieve a balance between isolation and performance. This is a significant finding, as it suggests that our snapshot-based isolation mechanism can be effectively utilized in real-world scenarios without incurring prohibitive performance penalties.

However, it is vital to note that the performance evaluation conducted is limited in scope. The choice to only measure execution time is an over-simplification, as the snapshotting process consumes significant system resources, particularly memory. Future work should include a more comprehensive performance evaluation, including metrics such as memory usage, CPU load, and I/O operations. This would provide a more holistic understanding of the performance implications of our snapshot-based isolation mechanism.

Additionally, it would be interesting to test what more directories it is possible to ignore to further optimize the snapshot creation time. Currently, only `/proc`, `/sys`, `/dev`, `/run`, and `/tmp` are being ignored during snapshot creation, meaning that every single other file is included. This makes it so the container is as close as possible to the baremetal environment, while also not creating such a heavy snapshot that the system is unusable. However, this also means that files such as `/root/.ssh/authorized_keys` are also available to the snapshot, which may not be desirable in certain scenarios. Exploring which directories can be safely ignored without impacting the functionality of the workflows would be a valuable avenue for future research.

In regard to isolation, the results indicate that our snapshot-based isolation mechanism is effective at preventing unauthorized modifications to both system and workflow files. There is the ability to read files, however, as mentioned previously, this can be managed by ignoring certain directories during snapshot creation. Additionally, the files that were changed inside the snapshot were not persistent, which is the desired behavior. It is also vital to note that the results indicate that, without isolation, it is possible to read and modify both system and workflow files, meaning that a malicious workflow could potentially compromise either other workflows, or the machine in which it is running. This highlights the importance of isolation in CI/CD environments, where untrusted code is often executed. The non-isolated system still had security settings turned on, such as the sandbox, but it is clearly not enough to prevent malicious activity.

However, it is vital to note that the isolation tests conducted are relatively basic. Future work should include more comprehensive isolation tests, including attempts to exploit known

vulnerabilities and bypass isolation mechanisms. This would provide a more robust evaluation of the security guarantees provided by our snapshot-based isolation mechanism.

In regard to RAM usage, the conclusions are similar to those of execution time. The most isolated system uses the most RAM, while the least isolated system uses the least RAM. However, the system that reuses the snapshot setup has RAM usage very close to that of the no isolation system. This further reinforces the conclusion that, with proper management of snapshots, it is possible to achieve a balance between isolation and performance. However, as with execution time, the RAM usage evaluation conducted is limited in scope. Future work should include a more comprehensive evaluation of resource usage.

Additionally, automating the snapshot management process would be a valuable enhancement to the system. Currently, the creation and management of snapshots is a manual process, which may not be practical in real-world scenarios. Developing an automated snapshot management system that can create and store snapshots as needed would greatly enhance the usability of the system.

Finally, there is the question of extending the system to support more complex workflows. Currently, the system is designed to execute simple shell scripts. However, real-world workflows are often more complex, involving multiple steps and dependencies. Extending the system to, perhaps, translate Jenkinsfiles into the appropriate shell scripts for execution within the snapshot would be a valuable avenue for future research.

The results presented in this work provide a strong foundation for the viability of snapshot-based isolation mechanisms in CI/CD environments, despite the limitations characteristic of an initial research work. They also present two modes of operation for different resource and time constraints. When time is not critical and sufficient resources are available, the fully isolated mode can be used for stronger isolation guarantees. When time is constrained, the reuse mode offers a better balance between isolation and performance. It is advisable to routinely create and verify fresh snapshots so that reused images remain up-to-date and secure. Although snapshot creation is slow, it can run in the background without affecting workflow performance and can be scheduled as frequently as security requirements demand.

VII. Conclusion

This paper argued for treating isolation as a foundational design principle in CI/CD systems rather than as an optional hardening mechanism layered on top of permissive execution models. We presented a snapshot-based execution approach that enforces strong isolation by construction, ensuring that each workflow runs in an ephemeral environment derived from a concrete and reproducible system snapshot. By making execution environments explicit artifacts and decoupling environment reuse from runtime state sharing, the proposed model strengthens security guarantees while preserving practical usability.

Through a prototype implementation integrated into a Jenkins-based CI/CD setup, we demonstrated that snapshot-based isolation can be realized using existing tooling and infrastructure. Our evaluation shows that while fully isolated executions incur significant setup overhead, reusing pre-generated snapshots enables execution times that are comparable to conventional, non-isolated workflows. Importantly, the isolation experiments confirm that snapshot-based execution effectively prevents persistent unauthorized modifications to system and workflow files, substantially limiting the impact of malicious or compromised workflows.

Overall, these results indicate that snapshot-based isolation offers a viable and effective foundation for securing modern CI/CD pipelines without sacrificing performance in steady-state operation. By shifting security guarantees from fragile configuration choices to architectural enforcement, this approach reduces the attack surface of CI/CD systems and improves their robustness against misconfiguration and supply-chain attacks. Future work will focus on refining environment capture, expanding isolation testing, and supporting more complex workflows, further advancing isolation-first CI/CD design as a practical and scalable security strategy.

References

[1] M. Shahin, M. Ali Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices."

[2] O. Elazhary, C. Werner, Z. S. Li, D. Lowlind, N. A. Ernst, and M.-A. Storey, "Uncovering the Benefits and Challenges of Continuous Integration Practices."

[3] S. M. Saleh, N. Madhavji, and J. Steinbacher, "A Systematic Literature Review on Continuous Integration and Deployment (CI/CD) for Secure Cloud Computing."

[4] N. Pecka, L. Ben Othmane, and A. Valani, "Privilege Escalation Attack Scenarios on the DevOps Pipeline Within a Kubernetes Environment."

[5] R. Carlos Bautista Ramos and S. G. Yoo, "Cybersecurity in DevOps Environments: A Systematic Literature Review."

[6] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the Security of Github CI Workflows." [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[7] H. Onsori Delicheh, A. Decan, and T. Mens, "Quantifying Security Issues in Reusable JavaScript Actions in GitHub Workflows."

[8] E. Riggio and C. Pautasso, "Pipelines Under Pressure: An Empirical Study of Security Misconfigurations of GitHub Workflows."

[9] M. Mangla, "Securing CI/CD Pipeline: Automating the detection of misconfigurations and integrating security tools," Master's thesis, Dublin, National College of Ireland, January 2023, submitted. [Online]. Available: https://norma.ncirl.ie/6529/

[10] G. Benedetti, L. Verderame, and A. Merlo, "Automatic Security Assessment of GitHub Actions Workflows."

[11] M. Freitas and L. Rocha, "GASH – The GitHub Actions Smell Hunter."

[12] Z. Pan, W. Shen, X. Wang, Y. Yang, R. Chang, Y. Liu, C. Liu, Y. Liu, and K. Ren, "Ambush From All Sides: Understanding Security Threats in Open-Source Software CI/CD Pipelines."

[13] M. Moazen, A. M. Ahmadian, and M. Balliu, "Granite: Granular Runtime Enforcement for GitHub Actions Permissions." [Online]. Available: https://arxiv.org/abs/2512.11602

[14] M. Schlegel and K.-U. Sattler, "Capturing end-to-end provenance for machine learning pipelines."

[15] P. Bajpai and A. Lewis, "Secure Development Workflows in CI/CD Pipelines."

[16] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight Virtualization for Serverless Applications ." [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/agache